

Ticket Management & Version Control Integration System

A Microservices-Based Project Paper

Group 10

Microservices Architecture Course
2024–2025 Academic Year

Project Topic	Ticket Management + Version Control
Technologies	Python (FastAPI), Docker, SQLite, REST
Services	User Service · Ticket Service · VCS Service
Inspiration	JIRA, GitHub, Asana, Confluence, Git
Document Type	Project Paper (Draft)

Abstract

This paper presents the design and implementation of a Ticket Management and Version Control Integration System built using a microservices architecture. The system addresses a critical gap in modern software development workflows: the disconnection between issue-tracking tools (such as JIRA or Asana) and version control systems (such as Git or GitHub). By enforcing a strict bidirectional relationship between tickets and commits — wherein a ticket cannot be closed without at least one linked commit, and every commit must reference a valid ticket — the system brings traceability, accountability, and process integrity to development teams. The platform comprises three independent microservices (User Service, Ticket Service, and Version Control Service) communicating exclusively via REST APIs, all containerised with Docker and orchestrated through Docker Compose. The implementation uses Python with the FastAPI framework and SQLite for persistence within each service boundary.

1. Introduction

In modern agile software development, two categories of tools are indispensable: issue-tracking systems and version control systems. Issue trackers (e.g., JIRA, Asana, Confluence) enable teams to plan, assign, and monitor tasks. Version control systems (e.g., Git, GitHub) record every code change with metadata such as author, timestamp, and a descriptive message. Despite being used side-by-side daily, these systems are typically separate, requiring developers to manually cross-reference commit messages with ticket identifiers — a process that is error-prone, inconsistent, and difficult to audit.

The Ticket Management & Version Control Integration System (TVCIS) solves this by building the relationship as a first-class constraint at the API layer. Three core capabilities are provided:

- Every code commit is required to carry a Ticket ID at creation time.
- A ticket can only transition to the DONE state when at least one commit is linked to it.
- Both perspectives — all commits for a ticket, and the ticket for any given commit — are instantly queryable.

This paper describes the motivation, architecture, implementation details, and integration design of TVCIS, deployed as a Docker-based microservices system.

2. Background

2.1 Ticket Management Systems

Ticket management systems, also known as issue trackers, are central to agile project management. They provide a structured way to capture work items — bugs, features,

tasks, improvements — and track them through a defined lifecycle.

JIRA (Atlassian) is the industry-leading issue tracker, widely used in enterprise environments. It supports custom workflows, sprint planning, epics, story points, and integration with development tools. Tickets move through statuses such as To Do, In Progress, In Review, and Done, with rules governing valid transitions.

Asana provides a more visual, task-board oriented experience suitable for both technical and non-technical teams. **Confluence** (also Atlassian) serves as a knowledge-management layer often used alongside JIRA for documentation. **Slack** is commonly integrated to deliver ticket notifications and status updates in real time to team channels.

2.2 Version Control Systems

Git is a distributed version control system that records snapshots of a repository's state at each commit. Each commit has a unique SHA-1 hash (e.g., a3f8c2d), a commit message, author information, a timestamp, and a pointer to its parent commit. Branches allow parallel lines of development to proceed without interference.

GitHub extends Git with a web-based hosting platform and collaboration features including pull requests, code review, and issue tracking. A widely adopted convention is to include the ticket identifier in the commit message or branch name (e.g., *fix/TICK-123-login-bug*), enabling automated cross-linking. However, this remains a convention — not a system-enforced rule.

2.3 The Integration Gap

In practice, developers often forget to reference ticket IDs in commit messages, or managers mark tickets as done without verifying code exists. This creates a disconnect: tickets lack traceability to code, and commit history lacks context about why changes were made. TVCIS addresses this by making the link a hard constraint enforced at the API boundary.

3. System Design

3.1 Microservices Architecture

The system follows a microservices pattern where each service owns its domain, data, and business rules. Services communicate synchronously via HTTP REST APIs. No shared database exists — each service has its own SQLite instance mounted via a Docker volume.

Service	Port	Responsibility	Depends On
User Service	8001	User creation, roles, lookup	—

Ticket Service	8002	Ticket CRUD, status transitions, enforce DONE rule	User Service, VCS Service
VCS Service	8003	Commit creation (ticket required), branch management, repo stats	Ticket Service, User Service

Table 1: Microservice summary

3.2 The Ticket–Commit Relationship

The core integration is implemented as two cross-service validation calls:

- **Commit creation (VCS Service → Ticket Service):** When a new commit is submitted, the VCS Service calls *GET /tickets/{ticket_id}* on the Ticket Service. If the ticket does not exist, the commit is rejected with HTTP 404.
- **Ticket closure (Ticket Service → VCS Service):** When a ticket status update to DONE is requested, the Ticket Service calls *GET /commits/by-ticket/{ticket_id}* on the VCS Service. If the result is an empty list, the update is rejected with HTTP 422 and a descriptive error payload.

Error response when closing a ticket without a commit:

```
{ "error": "DONE_REQUIRES_COMMIT", "message": "Ticket cannot be marked DONE
without at least one linked commit. Please commit your code first.",
  "ticket_id": "TICK-XXXXXXX", "linked_commits": 0 }
```

3.3 Ticket Status Lifecycle

Status	Description	Transition Rule
OPEN	Newly created, not yet assigned	Default; no rule
IN_PROGRESS	Actively being worked on	Free transition
IN_REVIEW	Code submitted, under review	Free transition
DONE	Completed and verified	Requires ≥1 linked commit

Table 2: Ticket status lifecycle and transition rules

4. Implementation

4.1 Technology Stack

Component	Technology	Rationale
Language	Python 3.11	Concise, widely supported in microservices
Web Framework	FastAPI	Async, OpenAPI auto-docs, Pydantic validation
Data Validation	Pydantic v2	Type-safe request/response models
HTTP Client	httpx	Async-compatible inter-service calls
Database	SQLite	Lightweight, no external dependency per service
Containers	Docker	Reproducible environments
Orchestration	Docker Compose	Multi-service startup, networking, volumes

Table 3: Technology stack

4.2 User Service

The User Service manages developer and manager accounts. Each user has an ID (UUID), username, email, full name, and role (developer, manager, or qa). The service persists data in SQLite and exposes standard CRUD endpoints. Usernames and emails must be unique; constraint violations return HTTP 409. The service runs on port 8001 and is the foundation both other services depend on for user validation.

4.3 Ticket Service

The Ticket Service is the heart of the system. Each ticket carries an auto-generated ID (format TICK-XXXXXXX), title, description, status, priority, assignee, reporter, and project label. The most critical logic lives in the PUT `/tickets/{id}` endpoint: when the requested status is DONE, the service makes a synchronous HTTP call to the VCS Service to count linked commits. If the count is zero, the update is rejected immediately with a human-readable error message. The service also exposes a `/tickets/{id}/summary` endpoint that aggregates ticket data, assignee information (from User Service), and linked commits (from VCS Service) into a single response, demonstrating the API composition pattern.

4.4 Version Control Service

The VCS Service simulates Git behaviour. On commit creation, a SHA-1 hash is generated from the commit message, author ID, and a UUID to ensure uniqueness. A 7-character short hash is derived for convenience. The `ticket_id` field is mandatory — if missing or blank, the request is rejected. The service then calls the Ticket Service to verify the ticket exists before persisting the commit. Additional features include branch management, lookup of a commit's parent ticket, and per-repository statistics (total commits, unique authors, tickets referenced, recent commits).

4.5 Docker Setup

Each service has its own Dockerfile based on `python:3.11-slim`. A single `docker-compose.yml` defines all three services on a shared bridge network (`ticketvcs-net`), with named volumes for data persistence. Health checks are defined for each service so that Docker waits for dependencies to be healthy before starting dependents. Environment variables are used to pass service URLs, enabling future deployment flexibility.

5. Conclusion

TVCIS demonstrates that the gap between ticket management and version control can be closed with relatively simple but carefully designed API constraints. By making the Ticket–Commit relationship a system-enforced rule rather than a convention, the platform eliminates entire categories of project management errors. The microservices approach ensures each domain remains independently deployable, scalable, and maintainable.

Future work could include a notification service (Slack/email alerts on ticket status changes), OAuth2 authentication, a web-based dashboard, and support for webhooks to integrate with real GitHub repositories.

References

- Atlassian. (2024). JIRA Software Documentation. <https://www.atlassian.com/software/jira>
- GitHub. (2024). GitHub Docs — Commits. <https://docs.github.com/en/repositories/committing-changes-to-your-project>
- FastAPI. (2024). FastAPI Documentation. <https://fastapi.tiangolo.com>
- Docker. (2024). Docker Compose Documentation. <https://docs.docker.com/compose>
- Fowler, M. (2014). Microservices — a definition of this new architectural term. martinfowler.com
- Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly Media.
- Richardson, C. (2018). Microservices Patterns. Manning Publications.